Lahore University of Management Sciences

School of Science and Engineering

AI600 – Deep Learning

Spring 2026

Assignment 1

Verda Batool 24280065

GitHub repository: AI600-PA1

Question 1: Feedforward Neural Networks using Tabular Data

Part A: Data Pre-processing and Exploratory Analysis

0.1 Dataset Structure

The original dataset had 41348 samples and it was split into training-val set using 80-20 ration. Hence the final training dataset consisted of 33,078 samples and 7 columns. Among these, 6 are input features and 1 is the target variable (`price_class`).

- **Categorical features:**
  - `neighbourhood_group` (string)
  - `room_type` (string)
- **Numerical features:**
  - `minimum_nights` (float64)
  - `amenity_score` (float64)
  - `number_of_reviews` (float64)
  - `availability_365` (float64)
- **Target variable:**
  - `price_class` (int64)

The dataset contains a mix of categorical and numerical variables. The target variable is discrete and represents four pricing categories (0–3), indicating that the task is a multi-class classification problem.

0.2 Handling Missing Values

We first quantified missing values across all features in the training dataset.

All features except the target variable contain missing values. However, the proportion of missing data in each feature is relatively small compared to the total dataset size (33,078 samples), therefore row deletion was not considered appropriate.

Feature	Missing Count
minimum_nights	1054
number_of_reviews	904
amenity_score	721
neighbourhood_group	671
room_type	485
availability_365	480
price_class	0

Table 1: Missing values per feature in the training dataset

Instead, each feature was analyzed individually and imputed using a strategy aligned with its distribution and statistical behavior.

Numerical Features

- `minimum_nights`: This feature contained 1054 missing values and exhibited extreme positive skewness (18.75). Descriptive statistics showed a median of 3 nights, a 75th percentile of 5 nights, and a maximum of 999 nights, indicating strong right-tail outliers. Mean imputation was therefore inappropriate due to sensitivity to extreme values.

  Group-wise median analysis across `neighbourhood_group` and `room_type` revealed only minor variation (2–3 nights). Since differences were negligible, global median imputation was selected for simplicity and robustness.

- `number_of_reviews`: This feature contained 904 missing values. To assess structural differences, average `availability_365` was compared between missing and non-missing groups (112 days vs. 100 days). The modest difference (approx. 12 days) did not indicate a distinct subgroup.

  Because listings can legitimately have zero reviews and the distribution is concentrated at low values, missing entries were interpreted as no recorded reviews. Median imputation would artificially inflate engagement metrics. Therefore, missing values were replaced with 0.

- `availability_365`: This variable (range 0–365 days) showed moderate positive skewness (0.78) and a bimodal distribution concentrated at 0 and 365 days. The feature contained 480 missing values.

  Comparative analysis showed minimal differences in mean `number_of_reviews` (3 reviews) and `price_class` between missing and non-missing groups, suggesting no structural separation.

  Since boundary values carry strong behavioral meaning, imputing with 0 or 365 would introduce bias. Global median imputation (43 days) was therefore selected as a neutral and robust approach.

- `amenity_score`: This feature demonstrated a strong monotonic relationship with `price_class`, indicating high predictive relevance. Missing values did not exhibit meaningful differences in review counts or price levels relative to non-missing entries.

  Given its importance and moderate skewness, global median imputation was applied to preserve robustness while maintaining predictive signal.

Categorical Features

- `neighbourhood_group`: This variable represents borough location and showed clear structural relevance to pricing (e.g., Manhattan listings had the highest average price class, while Bronx listings had the lowest). The dataset contained 671 missing values.

  Comparative analysis revealed negligible differences in average `price_class` and `amenity_score` between missing and non-missing groups. Imputing with the most frequent category would introduce geographic bias. Therefore, missing values were replaced with a new category labeled `"Unknown"`.

- `room_type`: This feature contained 611 missing values. Comparative analysis showed negligible differences in mean `price_class` and `amenity_score` between missing and non-missing entries.

  Rather than imputing with the modal category (which would distort class proportions), missing values were treated as a separate category (`"Unknown"`) prior to one-hot encoding.

Overall, this strategy:

- Minimizes information loss,
- Ensures robustness against skewed numerical distributions,
- Avoids artificial bias in categorical features,
- Preserves potentially informative missingness patterns.

0.3 Class Distribution of the Target Variable

The target variable `price_class` consists of four discrete categories (0–3). Table 2 summarizes the class frequencies and proportions in the training dataset.

price_class	Count	Percentage (%)
0	4454	13.47
1	18583	56.18
2	7908	23.91
3	2133	6.45

Table 2: Class distribution of the target variable

Figure 1 provides a visual representation of this distribution.

The dataset exhibits moderate class imbalance. Class 1 dominates the distribution (56.18%), while Class 3 accounts for only 6.45% of observations. Classes 0 and 2 represent 13.47% and 23.91%, respectively.

This imbalance has important modeling implications. A naive classifier predicting only the majority class (Class 1) would achieve over 56% accuracy without learning meaningful structure. Consequently, accuracy alone is not an adequate evaluation metric. Macro-averaged F1-score, balanced accuracy, or class-weighted loss functions should be considered to ensure fair performance across minority classes.

0.4 Relationships Between Individual Features and the Target Variable

To assess predictive relevance, we examined the relationship between each feature and `price_class` using visualization and correlation analysis.

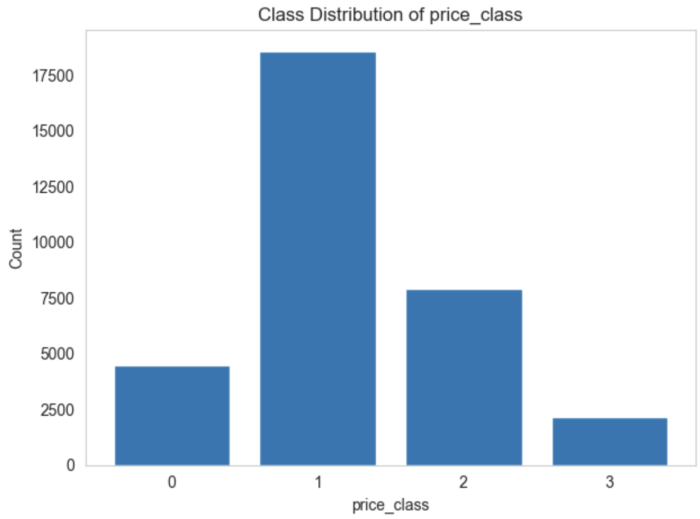

Figure 1: Bar chart showing distribution of `price_class`

0.4.1 Categorical Features

neighbourhood_group: The countplot reveals a strong structural association between geographic location and pricing:

- Manhattan contains a substantially higher proportion of listings in upper price classes (2 and 3).
- Brooklyn is concentrated primarily in mid-range classes (1 and 2).
- Queens, Bronx, and Staten Island are dominated by lower price classes (0 and 1).

Figure 2 shows the distribution of `price_class` across boroughs. This pattern is economically consistent with real estate market dynamics, where central and high-demand locations command higher prices. The clear stratification indicates that `neighbourhood_group` is a highly informative categorical predictor.

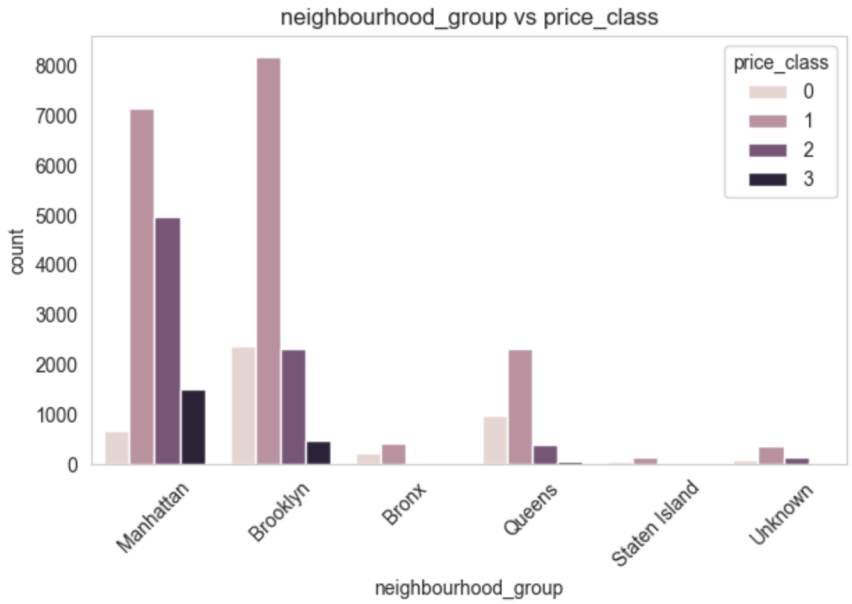

Figure 2: Distribution of price_class across neighbourhood_group categories.

room_type: A clear hierarchy emerges:

- Entire homes/apartments are disproportionately represented in higher price tiers.
- Private rooms occupy intermediate pricing levels.
- Shared rooms are concentrated in lower price categories.

The stratification across categories indicates that `room_type` carries strong predictive signal and is likely to contribute meaningfully to classification performance. Figure 3 illustrates the distribution of `price_class` across room types.

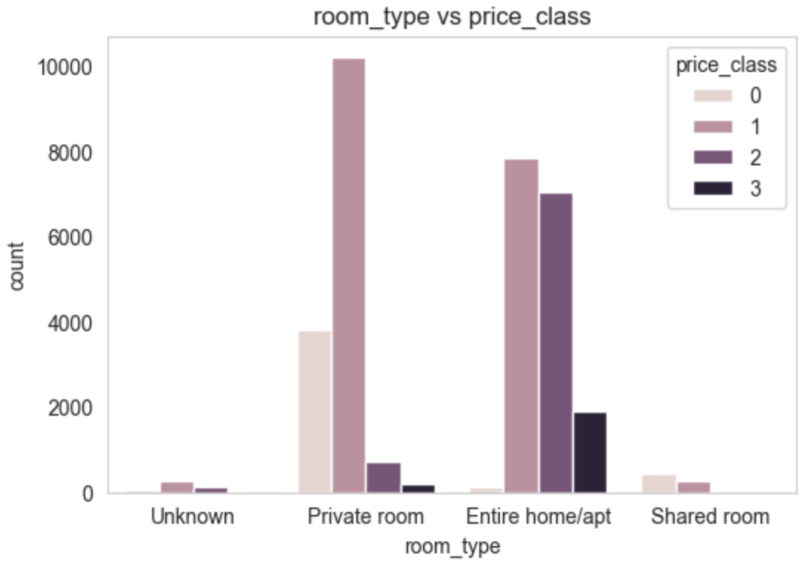

Figure 3: Distribution of price_class across room_type categories.

0.4.2 Numerical Features

minimum_nights: The boxplots show substantial overlap between categories and no monotonic separation. The distribution is heavily right-skewed with extreme outliers, and the Pearson correlation with `price_class` is negligible ($r \approx 0.02$). Figure 4 presents the distribution of `minimum_nights` across price classes. These results suggest that `minimum_nights` has minimal linear association with pricing tier and is unlikely to be a strong standalone predictor.

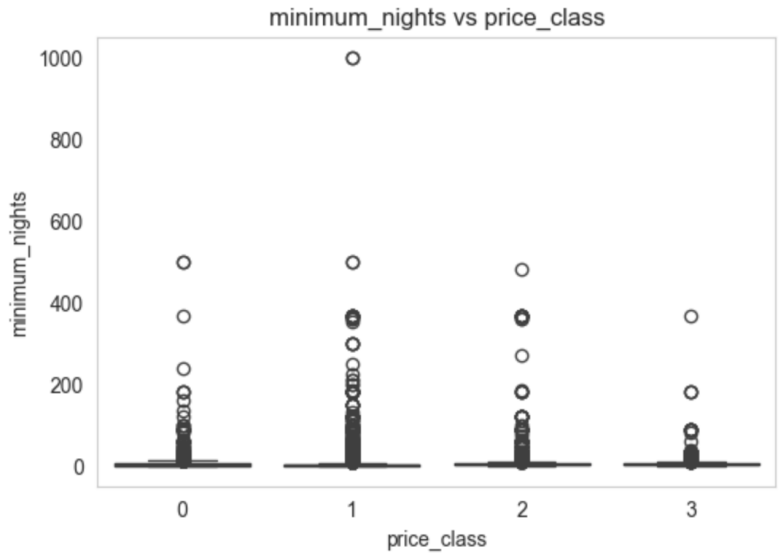

Figure 4: Boxplot of minimum_nights across price classes.

amenity_score: A clear monotonic increase is observed across price categories, with relatively limited overlap between adjacent classes. The correlation coefficient is very high ($r = 0.87$), indicating a strong positive relationship. Figure 5 shows the relationship between `amenity_score` and `price_class`. Among all numerical features, `amenity_score` exhibits the strongest association with the target variable and appears to be the most influential numerical predictor in the dataset.

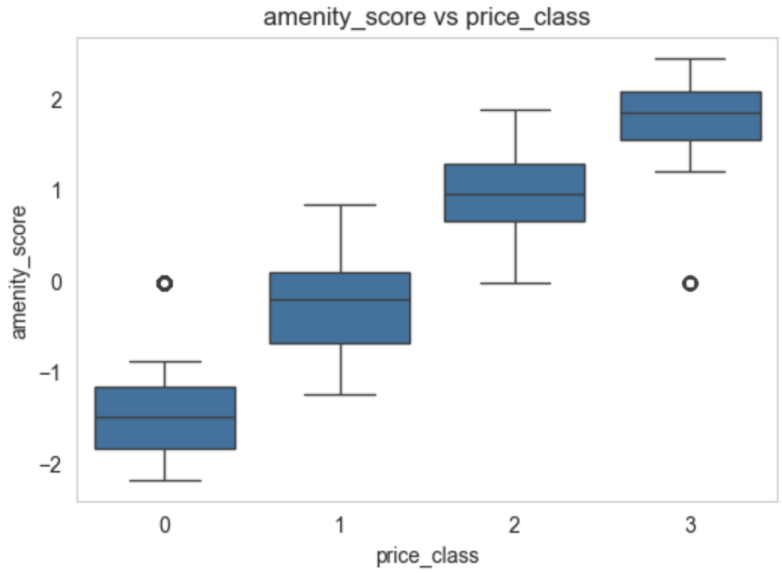

Figure 5: Boxplot of amenity_score across price classes.

number_of_reviews: The distributions are heavily right-skewed with significant overlap. Median values are similar across classes, and the correlation is near zero ($r \approx -0.03$). Figure 6 displays review counts across price classes. This indicates weak linear association and limited predictive strength when considered independently.

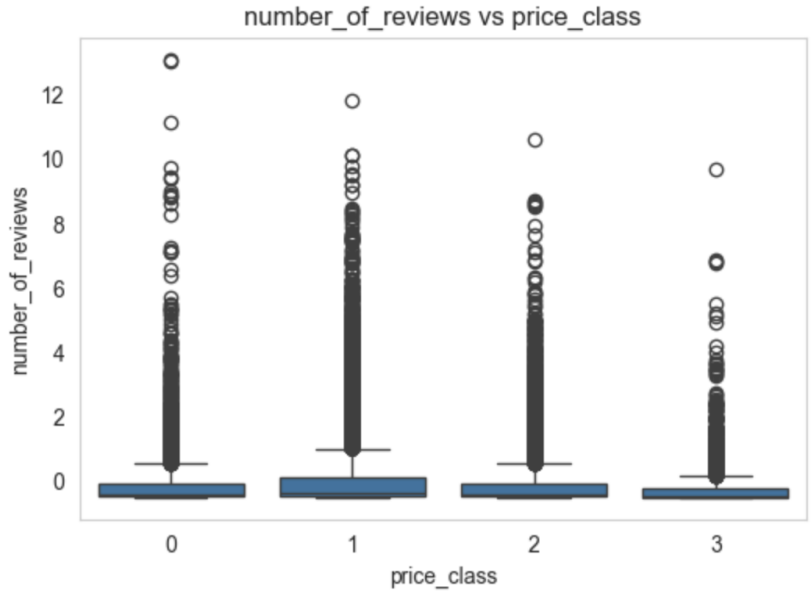

Figure 6: Boxplot of number_of_reviews across price classes.

availability_365: A moderate upward shift in median availability is observed for higher price classes, particularly Class 3. However, considerable overlap persists between categories, and the correlation remains weak ($r \approx 0.09$). Figure 7 presents availability across pricing tiers. Thus, `availability_365` contributes some signal but is not expected to be a dominant predictor.

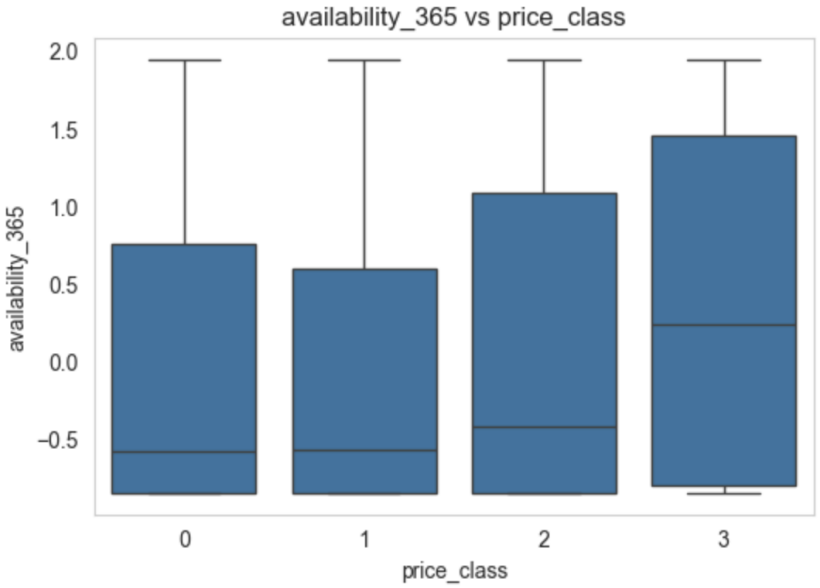

Figure 7: Boxplot of availability_365 across price classes.

0.5 Correlation Matrix Among Numerical Features

A correlation matrix was computed to examine linear relationships among numerical variables (Figure 8).

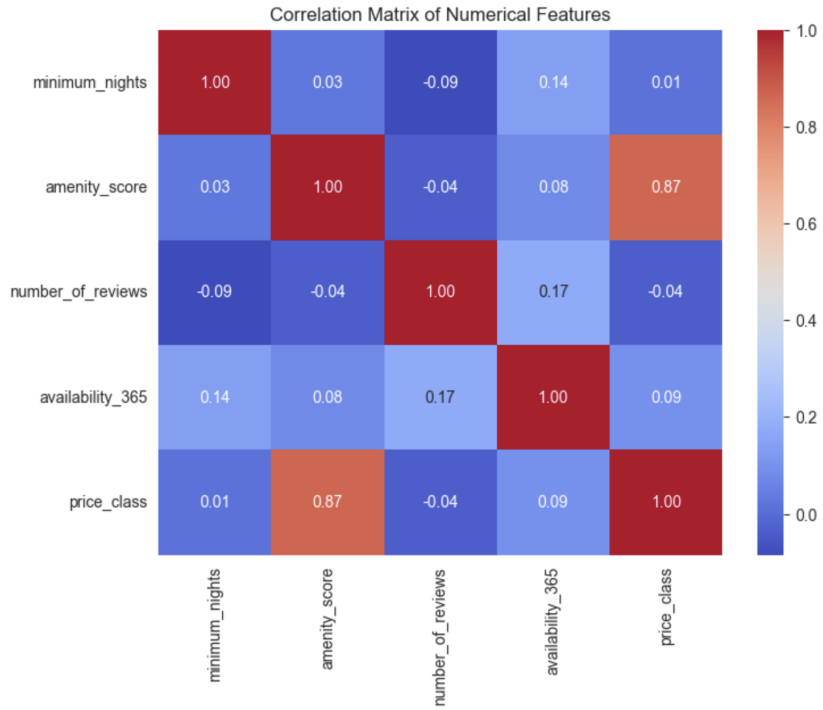

Figure 8: Correlation matrix of numerical features

The results indicate that correlations among predictor variables are weak ($|r| < 0.2$), suggesting low multicollinearity and minimal redundancy between numerical features.

Specifically:

- No pair of predictor variables exhibits strong linear association.
- There is no evidence of redundant numerical features.
- Feature removal based on multicollinearity is not necessary.

However, a strong positive correlation ($r = 0.87$) is observed between `amenity_score` and the target variable `price_class`. This magnitude is substantially higher than any other numerical association in the dataset.

Implications for Model Behavior

The exceptionally high correlation suggests that `amenity_score` will likely be the dominant numerical predictor in the feed-forward neural network.

In contrast, the remaining numerical variables exhibit weak linear relationships with the target:

- `availability_365`: weak positive correlation ($r \approx 0.09$)
- `minimum_nights`: negligible correlation ($r \approx 0.02$)
- `number_of_reviews`: near-zero correlation ($r \approx -0.03$)

These findings indicate that most predictive signal among numerical features is concentrated in `amenity_score`.

Expected Feature Influence

Based on visual analysis and correlation evidence, the expected influence hierarchy is:

- **Strong influence:**
  - `amenity_score`
  - `room_type`
  - `neighbourhood_group`
- **Moderate influence:**
  - `availability_365`
- **Low influence:**
  - `minimum_nights`
  - `number_of_reviews`

Potentially Dominant Feature

The correlation of 0.87 between `amenity_score` and `price_class` is unusually strong for real-world tabular data. While this likely reflects genuine structural importance (amenity quality influencing price tier), it also suggests that the neural network may rely heavily on this single feature during training.

Therefore, model evaluation should monitor whether performance is disproportionately driven by `amenity_score`, and confirm that no implicit target leakage exists in its construction.

0.6 Categorical Encoding

The categorical variables `neighbourhood_group` and `room_type` were encoded using one-hot encoding.

This method was selected because both variables are nominal and do not possess any inherent ordering. Applying label encoding would introduce artificial ordinal relationships (e.g., assigning increasing integers to categories), which could mislead the neural network during training.

Given the relatively small number of categories, the increase in dimensionality resulting from one-hot encoding is minimal and does not introduce excessive sparsity. This approach preserves categorical integrity while remaining computationally efficient.

0.7 Feature Normalization

All numerical features were normalized using Z-score standardization. For each feature, the mean was subtracted and the result divided by the standard deviation:

$$z = \frac{x - \mu}{\sigma}$$

This transformation centers each feature around zero and scales it to unit variance.

Normalization parameters (μ and σ) were computed using the training set only. The same transformation was subsequently applied to validation and test data to prevent data leakage.

Z-score standardization was chosen because feed-forward neural networks are sensitive to feature scale. Standardizing inputs ensures that all numerical features contribute proportionally during gradient-based optimization, improves convergence stability, and prevents features with larger magnitudes from disproportionately influencing the learning process.

Part B (a): Two-Layer Perceptron Implemented from Scratch

Model and notation. We implemented a fully-connected feed-forward neural network with two hidden layers using only NumPy, without automatic differentiation. Let $X \in \mathbb{R}^{d\times m}$ denote a batch of m training examples (columns), and let the one-hot labels be $Y \in \mathbb{R}^{K\times m}$ for K classes. The network has hidden widths h_1 and h_2, with parameters:

$$W_1 \in \mathbb{R}^{h_1\times d},\ b_1 \in \mathbb{R}^{h_1\times 1},\ W_2 \in \mathbb{R}^{h_2\times h_1},\ b_2 \in \mathbb{R}^{h_2\times 1},\ W_3 \in \mathbb{R}^{K\times h_2},\ b_3 \in \mathbb{R}^{K\times 1}.$$

We used the same architecture for both activation experiments, changing only the hidden-layer nonlinearity.

Forward propagation. For a given activation function $\phi(\cdot)$ applied in the hidden layers, the forward pass is:

$$\begin{aligned} Z_1 &= W_1X + b_1, \quad A_1 = \phi(Z_1), \\ Z_2 &= W_2A_1 + b_2, \quad A_2 = \phi(Z_2), \\ Z_3 &= W_3A_2 + b_3, \quad \hat{Y} = \mathrm{softmax}(Z_3), \end{aligned}$$

where the softmax is computed column-wise:

$$\hat{Y}_{k,j} = \frac{\exp(Z_{3,kj})}{\sum_{\ell=1}^{K}\exp(Z_{3,\ell j})}.$$

For numerical stability, we used the standard logit-shift trick by subtracting $\max_k Z_{3,kj}$ per column before exponentiation.

Cross-entropy loss. We trained using the multiclass cross-entropy loss averaged over the batch:

$$\mathcal{L} = -\frac{1}{m}\sum_{j=1}^{m}\sum_{k=1}^{K} Y_{k,j}\log(\hat{Y}_{k,j}).$$

To avoid $\log(0)$, $\hat{Y}$ was clipped by a small ϵ before computing the logarithm.

Backward propagation. Using the standard softmax + cross-entropy simplification, the gradient at the output logits is:

$$\frac{\partial\mathcal{L}}{\partial Z_3} = \hat{Y} - Y \quad \in \mathbb{R}^{K\times m}.$$

Then, by applying the chain rule backward through the layers:

$$\begin{aligned} \frac{\partial\mathcal{L}}{\partial W_3} &= \frac{1}{m}\left(\frac{\partial\mathcal{L}}{\partial Z_3}\right)A_2^\top, \quad \frac{\partial\mathcal{L}}{\partial b_3} = \frac{1}{m}\sum_{j=1}^{m}\left(\frac{\partial\mathcal{L}}{\partial Z_3}\right)_{:,j}, \\ \frac{\partial\mathcal{L}}{\partial A_2} &= W_3^\top\frac{\partial\mathcal{L}}{\partial Z_3}, \quad \frac{\partial\mathcal{L}}{\partial Z_2} = \left(\frac{\partial\mathcal{L}}{\partial A_2}\right)\odot\phi'(Z_2), \\ \frac{\partial\mathcal{L}}{\partial W_2} &= \frac{1}{m}\left(\frac{\partial\mathcal{L}}{\partial Z_2}\right)A_1^\top, \quad \frac{\partial\mathcal{L}}{\partial b_2} = \frac{1}{m}\sum_{j=1}^{m}\left(\frac{\partial\mathcal{L}}{\partial Z_2}\right)_{:,j}, \\ \frac{\partial\mathcal{L}}{\partial A_1} &= W_2^\top\frac{\partial\mathcal{L}}{\partial Z_2}, \quad \frac{\partial\mathcal{L}}{\partial Z_1} = \left(\frac{\partial\mathcal{L}}{\partial A_1}\right)\odot\phi'(Z_1), \\ \frac{\partial\mathcal{L}}{\partial W_1} &= \frac{1}{m}\left(\frac{\partial\mathcal{L}}{\partial Z_1}\right)X^\top, \quad \frac{\partial\mathcal{L}}{\partial b_1} = \frac{1}{m}\sum_{j=1}^{m}\left(\frac{\partial\mathcal{L}}{\partial Z_1}\right)_{:,j}. \end{aligned}$$

Finally, we updated parameters using vanilla batch gradient descent:

$$W_\ell \leftarrow W_\ell - \eta\frac{\partial\mathcal{L}}{\partial W_\ell}, \quad b_\ell \leftarrow b_\ell - \eta\frac{\partial\mathcal{L}}{\partial b_\ell} \quad \text{for } \ell \in \{1,2,3\},$$

where η is the learning rate.

Figure 9: Training and validation accuracy vs. iterations for the two-hidden-layer MLP using sigmoid and ReLU hidden activations.

Activation functions and gradient flow. We compared two hidden-layer activations:

Sigmoid: $\phi(z) = \sigma(z) = \frac{1}{1+e^{-z}}$ with derivative $\sigma'(z) = \sigma(z)(1-\sigma(z))$. Since $\sigma'(z) \leq 0.25$ and decays rapidly when $|z|$ is large, gradients can diminish as they backpropagate through multiple layers. This vanishing-gradient tendency can slow optimization and make training sensitive to initialization, feature scaling, and learning rate.

ReLU: $\phi(z) = \max(0, z)$ with derivative $\phi'(z) = \mathbb{I}[z > 0]$. For units with $z > 0$, the derivative is 1, which helps preserve gradient magnitude through depth and typically improves optimization dynamics. However, for $z \leq 0$, the derivative is 0, which can lead to "dead" units that stop learning if many activations remain negative.

Training procedure and results. We trained using vanilla batch gradient descent for at least 200 iterations for both activations. At each iteration, we recorded training and validation accuracy:

$$\text{Acc} = \frac{1}{m}\sum_{j=1}^{m} \mathbb{I}\left[\arg\max_k \hat{Y}_{k,j} = y_j\right].$$

The final accuracies were:

- **Sigmoid:** Train Acc = **83.03%**, Val Acc = **83.42%**
- **ReLU:** Train Acc = **83.15%**, Val Acc = **83.68%**

Figure 9 shows the training and validation accuracy as a function of iterations for both activation functions. Overall, ReLU exhibited faster convergence and more stable gradient flow, while sigmoid required more careful hyperparameter tuning due to its weaker gradient propagation in saturated regimes.

Stabilizing Sigmoid Optimization. Initial experiments with the sigmoid activation showed slow convergence and sensitivity to hyperparameters. This behavior is consistent with the theoretical properties of the sigmoid function. Since

$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \leq 0.25,$$

gradients are naturally attenuated during backpropagation. When the pre-activation values Z_ℓ become large in magnitude, sigmoid units enter saturation regimes where $\sigma'(z) \approx 0$, further weakening gradient flow to earlier layers.

To mitigate this issue and improve optimization, we applied the following adjustments:

- **Adjusted Initialization Scaling:** We rescaled initial weights to approximate Xavier-style scaling, ensuring that pre-activation magnitudes remain moderate in early training.
- **Higher Learning Rate:** Compared to ReLU, sigmoid required a larger learning rate to compensate for smaller gradient magnitudes and avoid excessively slow parameter updates.
- **Class Weighting:** To address class imbalance, weighted cross-entropy was used, preventing the network from collapsing to the majority class and improving effective gradient signals.

With these modifications, sigmoid achieved competitive performance, demonstrating that while it is more sensitive to optimization conditions, it can still learn effectively when gradients are properly conditioned.

Learning Rate Sensitivity of ReLU: For the ReLU activation, we observed that very small learning rates led to little or no convergence. In this regime, training accuracy remained nearly constant across iterations. This can be explained by the piecewise-linear nature of ReLU:

$$\phi(z) = \max(0, z), \quad \phi'(z) = \mathbb{I}[z > 0].$$

When gradients are small and the learning rate is too low, weight updates may be insufficient to significantly move parameters. As a result, the model may remain in a flat region of the loss surface (a shallow valley or plateau), causing extremely slow progress.

In contrast to sigmoid, ReLU preserves gradient magnitude for active units ($\phi'(z) = 1$), which typically enables faster convergence. However, effective optimization still depends on selecting a learning rate large enough to produce meaningful parameter updates. Thus, while ReLU improves gradient flow relative to sigmoid, it is not immune to poor hyperparameter selection.

The experiments confirm classical gradient-flow theory:

- Sigmoid suffers from gradient attenuation in deeper layers and requires careful scaling and learning-rate tuning.
- ReLU maintains stronger gradient propagation but can stagnate under excessively small learning rates.
- Proper conditioning (normalization, initialization, and learning-rate selection) is critical for stable optimization in manually implemented neural networks.

Part B (b): Gradient Magnitude Comparison Across Layers

To analyze gradient flow across depth, we computed the average magnitude of the weight gradients for the first and second hidden layers at each training iteration. Specifically, for each epoch t, we recorded:

$$g_1(t) = \text{mean}\left(\left|\frac{\partial \mathcal{L}}{\partial W_1}\right|\right), \quad g_2(t) = \text{mean}\left(\left|\frac{\partial \mathcal{L}}{\partial W_2}\right|\right).$$

These quantities measure the average strength of gradient signals reaching each hidden layer during backpropagation.

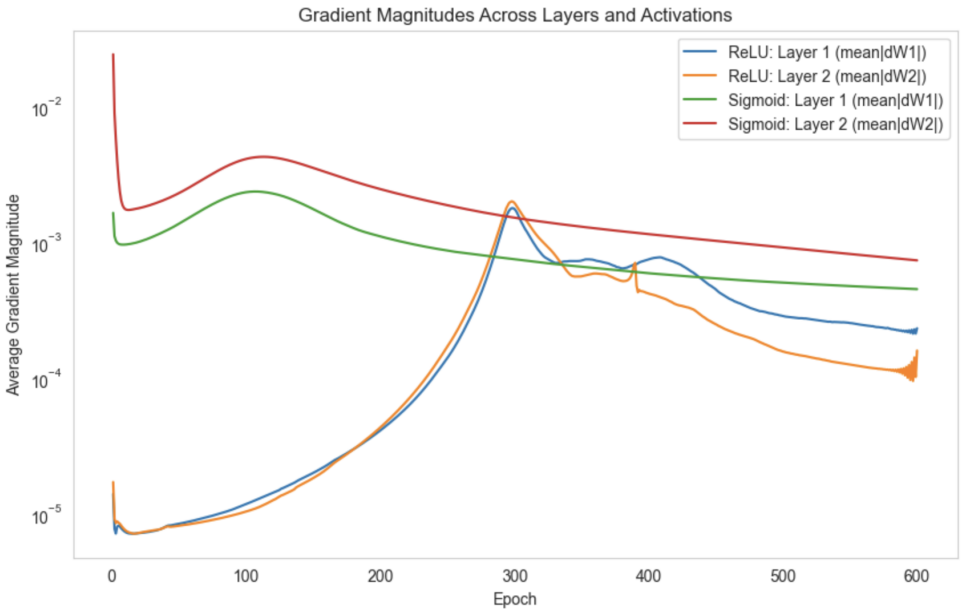

Figure 10: Average gradient magnitudes of the first and second hidden layer weights for ReLU and sigmoid activations (log scale).

Quantitative Results: The mean gradient magnitudes across all iterations were:

- **ReLU:**

$$\text{mean}(g_1) = 3.23 \times 10^{-4}, \quad \text{mean}(g_2) = 2.69 \times 10^{-4}$$
$$\text{Layer ratio } \frac{g_1}{g_2} = 1.31$$

- **Sigmoid:**

$$\text{mean}(g_1) = 9.62 \times 10^{-4}, \quad \text{mean}(g_2) = 1.89 \times 10^{-3}$$
$$\text{Layer ratio } \frac{g_1}{g_2} = 0.52$$

Figure 10 shows the evolution of these gradient magnitudes over training iterations (log-scale).

For ReLU, the gradient magnitudes of the first and second hidden layers remain comparable throughout training. The average ratio $g_1/g_2 \approx 1.31$ indicates that gradient signals reaching the earlier layer are not attenuated relative to deeper layers. This reflects strong gradient preservation across depth. The curves also show relatively stable decay patterns over iterations, indicating consistent learning dynamics.

For sigmoid, although the absolute gradient magnitudes are initially larger than those of ReLU, a clear imbalance is observed across layers. The ratio $g_1/g_2 \approx 0.52$ indicates that gradients reaching the first hidden layer are approximately half the magnitude of those in the second layer. This attenuation is consistent with the theoretical behavior of sigmoid derivatives:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25,$$

which causes gradients to shrink multiplicatively as they propagate backward through layers.

Implications for Gradient Flow and Learning Dynamics. These results empirically confirm classical gradient-flow theory in deep networks:

- **ReLU preserves gradient magnitude across depth.** Since $\phi'(z) = 1$ for active units, gradients propagate more directly through layers, enabling faster and more stable optimization.
- **Sigmoid exhibits gradient attenuation.** Even when training succeeds, gradient magnitudes diminish as they propagate toward earlier layers, leading to weaker updates in shallow layers and greater sensitivity to initialization and learning-rate selection.
- **Gradient imbalance affects optimization speed.** Stronger preservation of gradient magnitude (as observed in ReLU) correlates with more stable and efficient convergence, whereas attenuated gradients (as observed in sigmoid) require additional conditioning to achieve comparable performance.

Overall, the quantitative comparison of g_1 and g_2 across iterations demonstrates that ReLU provides more effective gradient flow in deeper architectures, while sigmoid inherently weakens gradient signals as depth increases. These findings align with both theoretical expectations and empirical training behavior observed in Part B(a).

Part C (a): Gradient-Based Feature Attribution (Analytical)

For a two-hidden-layer neural network with input $\mathbf{x} \in \mathbb{R}^d$:

$$\begin{aligned} \mathbf{z}_1 &= W_1\mathbf{x} + \mathbf{b}_1, & \mathbf{a}_1 &= \phi(\mathbf{z}_1) \\ \mathbf{z}_2 &= W_2\mathbf{a}_1 + \mathbf{b}_2, & \mathbf{a}_2 &= \phi(\mathbf{z}_2) \\ \mathbf{z}_3 &= W_3\mathbf{a}_2 + \mathbf{b}_3, & \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z}_3) \end{aligned}$$

The cross-entropy loss is:

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

We compute $\dfrac{\partial \mathcal{L}}{\partial \mathbf{x}}$ using backpropagation. Loss is back propagated till input via output layer and two hidden layers.

$$\text{Input} \rightarrow \text{Hidden Layer 1} \rightarrow \text{Hidden Layer 2} \rightarrow \hat{y} \rightarrow \mathcal{L}$$

For Output Layer

For softmax combined with cross-entropy:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_3} = \hat{\mathbf{y}} - \mathbf{y}$$

Hence,

$$\boldsymbol{\delta}_3 = \hat{\mathbf{y}} - \mathbf{y}$$

For Second Hidden Layer

Through chain rule, we know that:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_3} \frac{\partial \mathbf{z}_3}{\partial \mathbf{a}_2}$$

Since

$$\mathbf{z}_3 = W_3 \mathbf{a}_2 + \mathbf{b}_3,$$

we have

$$\frac{\partial \mathbf{z}_3}{\partial \mathbf{a}_2} = W_3$$

Therefore,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = W_3^\top \frac{\partial \mathcal{L}}{\partial \mathbf{z}_3}$$

We calculated above,

$$\boldsymbol{\delta}_3 = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_3},$$

Hence,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_2} = W_3^\top \boldsymbol{\delta}_3.$$

For First Hidden Layer

We now propagate gradients from the second hidden layer to the first hidden layer.

Applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} \frac{\partial \mathbf{z}_2}{\partial \mathbf{a}_1}$$

Since

$$\mathbf{z}_2 = W_2 \mathbf{a}_1 + \mathbf{b}_2,$$

we have

$$\frac{\partial \mathbf{z}_2}{\partial \mathbf{a}_1} = W_2$$

Therefore,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = W_2^\top \frac{\partial \mathcal{L}}{\partial \mathbf{z}_2}$$

Defining

$$\boldsymbol{\delta}_2 = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_2},$$

we obtain

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} = W_2^\top \boldsymbol{\delta}_2.$$

Next, since the activation is applied elementwise:

$$\mathbf{a}_1 = \phi(\mathbf{z}_1),$$

we again apply the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_1} \odot \phi'(\mathbf{z}_1)$$

Thus,

$$\boldsymbol{\delta}_1 = \left(W_2^\top \boldsymbol{\delta}_2\right) \odot \phi'(\mathbf{z}_1).$$

Gradient with Respect to the Input

We will now propagate gradients from the first hidden layer to the input $\mathbf{x}$.

Applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_1} \frac{\partial \mathbf{z}_1}{\partial \mathbf{x}}$$

Since

$$\mathbf{z}_1 = W_1 \mathbf{x} + \mathbf{b}_1,$$

we have

$$\frac{\partial \mathbf{z}_1}{\partial \mathbf{x}} = W_1.$$

Therefore,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = W_1^\top \frac{\partial \mathcal{L}}{\partial \mathbf{z}_1}.$$

Defining

$$\boldsymbol{\delta}_1 = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_1},$$

we obtain the final expression:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = W_1^\top \boldsymbol{\delta}_1.$$

Final Expression

Substituting:

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = W_1^\top \left[\left(W_2^\top \left[\left(W_3^\top (\hat{\mathbf{y}} - \mathbf{y})\right) \odot \phi'(\mathbf{z}_2)\right]\right) \odot \phi'(\mathbf{z}_1)\right]} \tag{1}$$

Activation Derivatives

For sigmoid activation:

$$\phi'(z) = \sigma(z)(1 - \sigma(z))$$

For ReLU activation:

$$\phi'(z) = \mathbb{I}[z > 0]$$

Algorithm: Gradient-Based Feature Attribution **Input:** Trained parameters $W_1, b_1, W_2, b_2, W_3, b_3$, dataset $X \in \mathbb{R}^{d \times m}$, labels $Y \in \mathbb{R}^{K \times m}$, activation function $\phi(\cdot)$.

Output: Ranked list of input features by importance.

```
1. Initialize gradient accumulator G of size d with zeros.

2. For each sample j = 1 to m:

   # Forward pass
   z1 = W1 x_j + b1
   a1 = phi(z1)

   z2 = W2 a1 + b2
   a2 = phi(z2)

   z3 = W3 a2 + b3
   y_hat = softmax(z3)

   # Compute output layer gradient
   delta3 = y_hat - y_j

   # Backpropagate through hidden layer 2
   delta2 = (W3^T delta3)  phi'(z2)

   # Backpropagate through hidden layer 1
   delta1 = (W2^T delta2)  phi'(z1)

   # Gradient w.r.t. input features
   grad_x = W1^T delta1

   # Accumulate absolute gradient magnitudes
   G = G + |grad_x|

3. Compute average importance score:
   importance = G / m

4. Rank features in descending order of importance.
```

This is the non-vectorized variant of the algorithm.

Justification of Gradient-Based Feature Importance From an optimization perspective, the quantity

$$\frac{\partial \mathcal{L}}{\partial x_i}$$

measures the local sensitivity of the loss function with respect to input feature x_i. By definition of the gradient, (as discussed by sir in class) a first-order Taylor approximation gives

$$\mathcal{L}(\mathbf{x} + \Delta\mathbf{x}) \approx \mathcal{L}(\mathbf{x}) + \sum_i \frac{\partial \mathcal{L}}{\partial x_i} \Delta x_i.$$

Thus, the magnitude

$$\left| \frac{\partial \mathcal{L}}{\partial x_i} \right|$$

quantifies how strongly a small perturbation in feature x_i affects the loss. If this magnitude is large, even a small change in x_i produces a significant change in the objective function, indicating that the feature has strong influence on model predictions.

Averaging this quantity across samples provides a global measure of feature importance. Therefore, features with larger average gradient magnitudes exert greater influence on the optimization landscape and contribute more strongly to the learned decision function.

Part C (b): Implementation and Results

The gradient-based attribution scores identify features to which the model's predicted class score is most sensitive. A large value of $|\partial o_m/\partial x_i|$ means that a small perturbation in feature x_i produces a large change in the winning-class score, indicating that the trained decision function relies strongly on that feature.

Feature	**Importance Score S_i**
amenity_score	2.3878
room_type_Private room	0.5857
room_type_Shared room	0.4674
neighbourhood_group_Manhattan	0.3945
room_type_Unknown	0.1296
minimum_nights	0.1289
availability_365	0.0927
neighbourhood_group_Unknown	0.0851
neighbourhood_group_Brooklyn	0.0727
neighbourhood_group_Staten Island	0.0699
number_of_reviews	0.0476
neighbourhood_group_Queens	0.0443

Table 3: Gradient-based feature importance using winning-logit sensitivity (Aggarwal, Section 2.8). Scores represent average absolute input-gradient magnitude aggregated over correctly classified training samples.

These attributions complement the analysis in Part A by providing a local, model-based explanation of feature influence. In Part A, feature effects were inferred using correlation matrix, whereas here the ranking reflects the learned nonlinear decision surface of the MLP. Features that were strongly associated with price class in Part A are expected to appear high in the gradient ranking.

Algorithm: Gradient-based feature attribution (winning-logit sensitivity)

```
Input:
    Trained neural network (W1, W2, W3, etc.)
    Training dataset X, labels Y
    Flag only_correct (True/False)

Output:
    Ranked list of input features by importance

Steps:

1. Initialize importance score vector S of size d (number of input features) to zeros.
```

```
2. For each training sample (x, y):

    a. Perform forward pass through the network.
    b. Compute predicted class y_hat.

    c. If only_correct is True and y_hat != y:
           Skip this sample.

    d. Compute gradient of the loss with respect to input:
           g = L / x

    e. Take absolute value of gradient:
           |g|

    f. Add |g| to S (element-wise accumulation).

3. Divide S by the number of aggregated samples
   (to obtain average magnitude).

4. Rank features in descending order of S.

5. Return ranked feature list.
```

Part D: Test Evaluation and Generalization Analysis

We evaluated both trained models (ReLU and Sigmoid activations) on the held-out test dataset after applying the identical preprocessing pipeline used during training (categorical encoding, feature alignment, and normalization using training statistics).

Performance Summary

- **ReLU:** Train Accuracy = 0.7778, Validation Accuracy = 0.7831, Test Accuracy = 0.4344
- **Sigmoid:** Train Accuracy = 0.8298, Validation Accuracy = 0.8349, Test Accuracy = 0.5631

Analysis of Generalization Behavior

The ReLU-based network exhibits consistently low performance across training and validation, suggesting limited representational learning under the chosen hyperparameters. Its test accuracy (0.4344) further declines, indicating poor generalization capacity.

In contrast, the Sigmoid-based network achieves substantially higher training and validation accuracy (approximately 0.83), suggesting effective optimization on the training distribution. However, its test accuracy drops to 0.5631, revealing a significant generalization gap of approximately 27 percentage points.

This discrepancy indicates distribution shift between training/validation data and the test data. While the model fits the training distribution well, it fails to maintain predictive performance under altered feature distributions in the test set.

Root Cause of Generalization Failure

From Part A (Exploratory Data Analysis), we observed class imbalance and strong correlations between certain features (notably *amenity_score*, room type, and borough indicators) and the

target label.

From Part C (Gradient-Based Feature Attribution), we found that the network relies heavily on a small subset of dominant features, particularly *amenity_score*. The magnitude of the input gradients revealed disproportionately large sensitivity to this feature relative to others.

This high feature dominance implies that even moderate shifts in the distribution of these influential features can cause substantial changes in logits and class predictions. Therefore, the model is highly sensitive to covariate shift.

Why Training Performance Is High but Test Performance Drops

The validation set is likely drawn from the same distribution as the training data, resulting in similar performance. However, the test set appears to exhibit distributional differences in the dominant features. Because the model learned sharp decision boundaries based on a narrow subset of highly predictive features, it lacks robustness to such shifts.

The Sigmoid network, while achieving higher accuracy, still demonstrates over-reliance on specific features and insufficient invariance to distributional changes.

Mitigation Strategy

To improve generalization in practice, we recommend:

- Incorporating regularization (e.g., L2 weight decay or dropout) to reduce sharp decision boundaries.
- Applying class-balanced training or weighted loss to mitigate imbalance bias.
- Performing cross-distribution validation to detect covariate shift early.
- Encouraging feature diversity through feature dropout or noise injection.

These strategies aim to reduce over-dependence on dominant features and improve robustness to distributional variation.

Question 2: Bias Gradients, Parameter Sharing, and Activation Effects

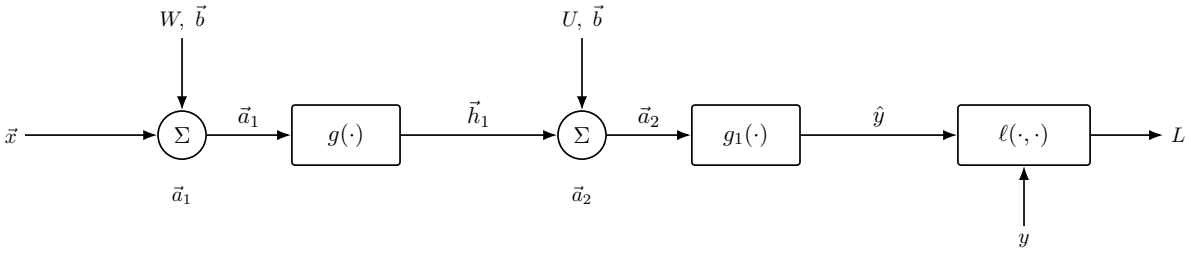

Figure 11: Computation graph for a two-layer MLP with a shared bias parameter.

Shared Bias Gradient

Assume $m = k$ so the same $\vec{b} \in \mathbb{R}^m$ is added in both affine transformations.

$$\vec{a}_1 = W\vec{x} + \vec{b}$$
$$\vec{h}_1 = g(\vec{a}_1)$$
$$\vec{a}_2 = U\vec{h}_1 + \vec{b}$$
$$\hat{y} = g_1(\vec{a}_2)$$
$$L = \ell(\hat{y}, y)$$

There are two paths through which the bias affects L:

Via output layer:

$$\vec{b} \to \vec{a}_2 \to L$$

Via hidden layer:

$$\vec{b} \to \vec{a}_1 \to \vec{h}_1 \to \vec{a}_2 \to L$$

Since there are two paths, the gradient contributions add:

$$\frac{\partial L}{\partial \vec{b}} = \underbrace{\frac{\partial L}{\partial \vec{a}_1}\frac{\partial \vec{a}_1}{\partial \vec{b}}}_{\text{via } \vec{a}_1} + \underbrace{\frac{\partial L}{\partial \vec{a}_2}\frac{\partial \vec{a}_2}{\partial \vec{b}}}_{\text{via } \vec{a}_2}$$

We know that

$$\vec{a}_1 = W\vec{x} + \vec{b}, \qquad \vec{a}_2 = U\vec{h}_1 + \vec{b}.$$
$$\frac{\partial \vec{a}_1}{\partial \vec{b}} = I, \qquad \frac{\partial \vec{a}_2}{\partial \vec{b}} = I,$$

So,

$$\frac{\partial L}{\partial \vec{b}} = \frac{\partial L}{\partial \vec{a}_1} + \frac{\partial L}{\partial \vec{a}_2}$$

where

$$\frac{\partial L}{\partial \vec{a}_2} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial \vec{a}_2}.$$

Here, $\frac{\partial L}{\partial \hat{y}}$ depends on the chosen loss function, while $\frac{\partial \hat{y}}{\partial \vec{a}_2}$ depends on the activation function g_1.

Since

$$\hat{y} = g_1(\vec{a}_2), \quad \Rightarrow \quad \frac{\partial \hat{y}}{\partial \vec{a}_2} = g_1'(\vec{a}_2).$$

Therefore,

$$\boxed{\frac{\partial L}{\partial \vec{a}_2} = \frac{\partial L}{\partial \hat{y}} \odot g_1'(\vec{a}_2)}$$

where $\odot$ denotes element-wise multiplication.

Now we compute $\dfrac{\partial L}{\partial \vec{a}_1}$.

The variable $\vec{a}_1$ affects the loss through the chain

$$\vec{a}_1 \to \vec{h}_1 \to \vec{a}_2 \to \hat{y} \to L.$$

By the chain rule,

$$\frac{\partial L}{\partial \vec{a}_1} = \frac{\partial L}{\partial \vec{a}_2}\,\frac{\partial \vec{a}_2}{\partial \vec{h}_1}\,\frac{\partial \vec{h}_1}{\partial \vec{a}_1}.$$

Now compute each factor:

$$\frac{\partial L}{\partial \vec{a}_2} = \vec{\delta}_2$$

$$\vec{a}_2 = U\vec{h}_1 + \vec{b} \quad \Rightarrow \quad \frac{\partial \vec{a}_2}{\partial \vec{h}_1} = U$$

$$\vec{h}_1 = g(\vec{a}_1) \quad \Rightarrow \quad \frac{\partial \vec{h}_1}{\partial \vec{a}_1} = g'(\vec{a}_1)$$

Substituting these into the chain-rule expression:

$$\frac{\partial L}{\partial \vec{a}_1} = \left(U^\top \vec{\delta}_2\right) \odot g'(\vec{a}_1).$$

we obtain the final expression

$$\boxed{\frac{\partial L}{\partial \vec{a}_1} = \left(U^\top \vec{\delta}_2\right) \odot g'(\vec{a}_1)}.$$

Thus, when the bias is shared across layers, its gradient equals the sum of the gradient contributions from both computational paths.

Optimization and Convergence Effect:
The shared-bias model has fewer degrees of freedom than the independent-bias model. This reduction in flexibility can slow convergence if the optimal solution of the unconstrained problem satisfies $b_1 \neq b_2$.

Consider the independent-bias objective:

$$J(W, U, b_1, b_2) = \ell(g_1(U\, g(Wx + b_1) + b_2)\,,\, y)\,.$$

The shared-bias model is obtained by enforcing $b_1 = b_2 = b$. Define

$$\tilde{J}(W, U, b) = J(W, U, b, b).$$

The shared-bias optimization problem can therefore be written as

$$\min_{W,U,b} \tilde{J}(W, U, b).$$

Equivalently, it can be expressed as the constrained version of the independent-bias problem:

$$\min_{W,U,b_1,b_2} J(W, U, b_1, b_2) \quad \text{s.t.} \quad b_1 = b_2.$$

Thus, bias sharing imposes the constraint $b_1 = b_2$, reducing the feasible parameter space and degrees of freedon. We are no longer free to move independently along each axis and the optimization is restricted to $b_1 = b_2$, hence, the shared-bias model must converge to a suboptimal point within the restricted constraint set, which can slow convergence and alter optimization stability. As a result, the shared-bias gradient becomes $\nabla_b \tilde{J} = \nabla_{b_1} J + \nabla_{b_2} J$, accumulating contributions from multiple computational paths. This summation can increase the gradient magnitude and the effective curvature, making updates more sensitive to the learning rate and potentially slowing or destabilizing convergence when the gradient contributions conflict.

1 Use of AI Tools and External Resources

External Learning Resources

For understanding and implementing a feed-forward neural network from scratch, the following publicly available educational videos were consulted for conceptual guidance:

- `https://www.youtube.com/watch?v=w8yWXqWQYmU`
- `https://www.youtube.com/watch?v=A83BbHFoKb8`

These resources were used strictly for conceptual clarification regarding forward propagation, backpropagation mechanics, and network structure. All implementation code was written independently in NumPy.

Use of Generative AI Assistance

ChatGPT (OpenAI) was used in the following ways:

- To refine analytical explanations. Bullet-point insights derived from experimental results were provided, and the model was used to structure them into coherent academic prose.
- To assist in debugging optimization issues encountered during training, particularly vanishing gradient behavior when using the Sigmoid activation function. Based on the discussion, weight scaling (Xavier-style initialization) and class-weighted cross-entropy loss were implemented to stabilize training.
- To generate LaTeX formatting for analytical derivations, pseudo-code, and report sections after handwritten derivations and algorithmic notes were uploaded.

The AI tool was used as an explanatory and formatting aid. All experimental design decisions, implementation logic, gradient derivations, and result interpretation was performed by me. AI assistance was limited to clarification, debugging guidance, and report composition support.